Lenguajes de Programación G2 2026-2

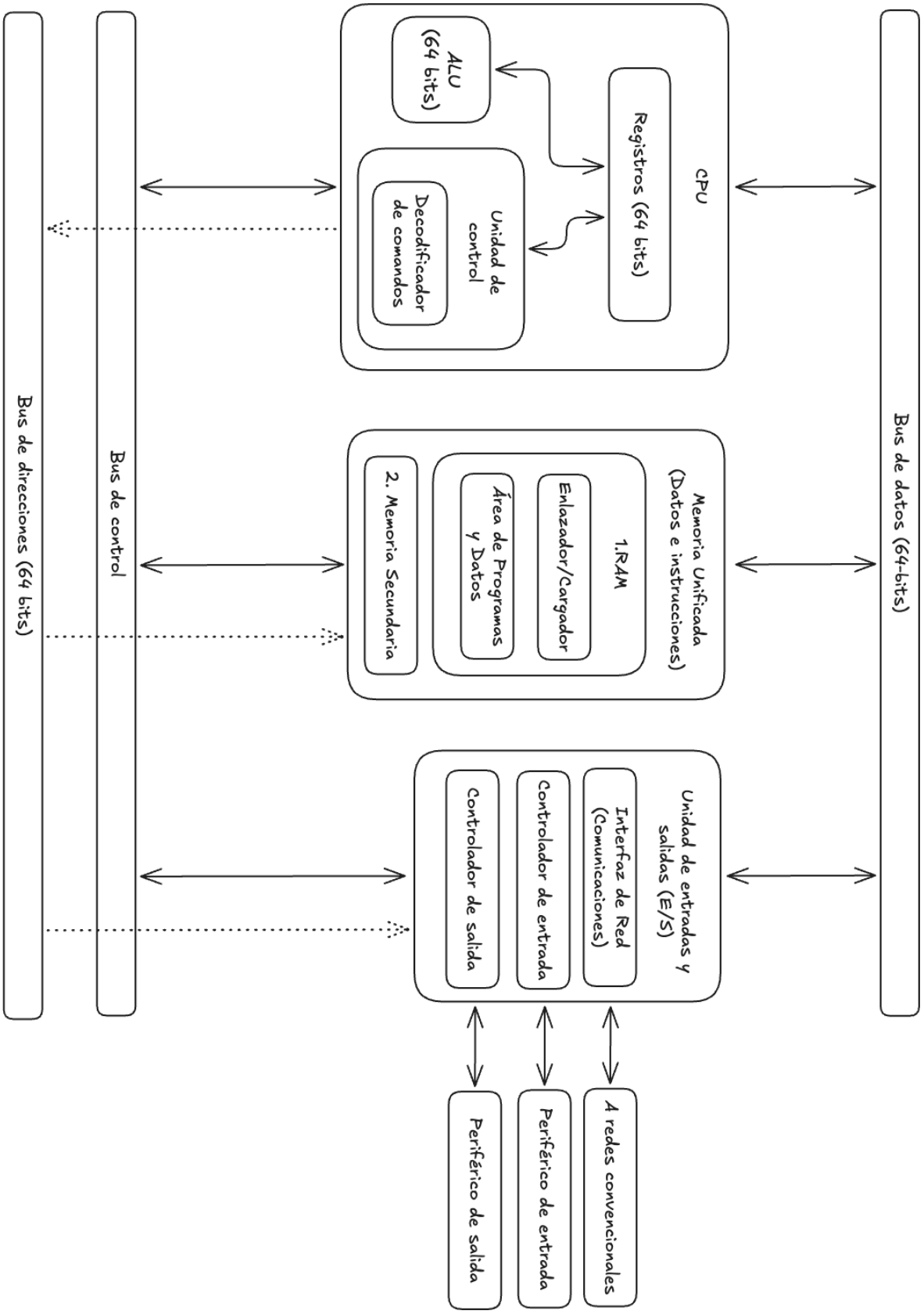
Tarea 9 - Diseño de un computador propio inspirado en el computador del Prof. Peña.

Integrantes Grupo A:

1.Juan Sebastian Umaña Camacho
2.Tomas Felipe Garzón Gómez
3.Michael Stiven Betancourt Gelves
4.Deibyd Santiago Barragán Gaitán
5.Maicol Sebastian Olarte Ramirez
6.Juan Luis Vergara Novoa
7.Alejandro Argüello Muñoz

Nombre de la empresa: Noctua Systems
Nombre del computador: Enigma-64
Arquitectura: Von Neumann
Tamaño de las palabras: 64 bits
Endiannes: Big - Endiann

Diagrama de arquitectura John von Neumann



Diseño de memoria

1. Decisiones base

Decisión	**Valor propuesto**	**Justificación**
Tamaño de palabra	64 bits (8 bytes)	Lo exige el enunciado ("al menos 64 bits") y coincide con el ancho del bus de datos: una palabra se transfiere en un solo ciclo de bus.
Unidad direccionable	Byte (byte-addressable)	Es obligatorio porque el requisito de instrucción es de longitud variable: si las instrucciones miden 2, 3, 5... bytes, el PC debe poder apuntar a cualquier byte. Con direccionamiento por palabra sería imposible.
Espacio direccionable	2^{64} bytes = 16 EiB (lógico) Exabytes	Lo determina el bus de direcciones de 64 bits.
Memoria física implementada	4 GiB (0x00000000 – 0xFFFFFFFF)	Solo se implementan los 32 bits bajos; los 32 altos deben ser cero. Es realista y suficiente para el emulador.
Ordenamiento de bytes	Big-endian	El byte más significativo va en la dirección menor. Para esta materia es la mejor opción porque la traducción manual a lenguaje de máquina (tarea de la Persona 7) se lee igual que se escribe en papel.

2. Alineación:

Los datos deben estar alineados a su tamaño natural (una palabra de 64 bits solo puede empezar en direcciones múltiplo de 8). Las **instrucciones son la excepción**: se alinean a byte, porque son de longitud variable. Si un acceso a datos está desalineado, la unidad de memoria genera una excepción de alineación.

3. Uso de Cache:

El grupo decidió no incorporar caché por el momento ya que la caché es transparente al programador y no ocupa direcciones del mapa de memoria; se ubica físicamente entre la CPU y los buses. Su incorporación posterior no modifica el mapa de memoria ni el ISA (Instruction set architecture).

4. *Organización física de la RAM:*

4 GiB = 2^{32} bytes = 2^{29} palabras de 64 bits. Se organiza en 8 bancos de 8 bits de ancho que se leen en paralelo, de modo que una sola dirección entrega los 64 bits del bus de datos en un ciclo.

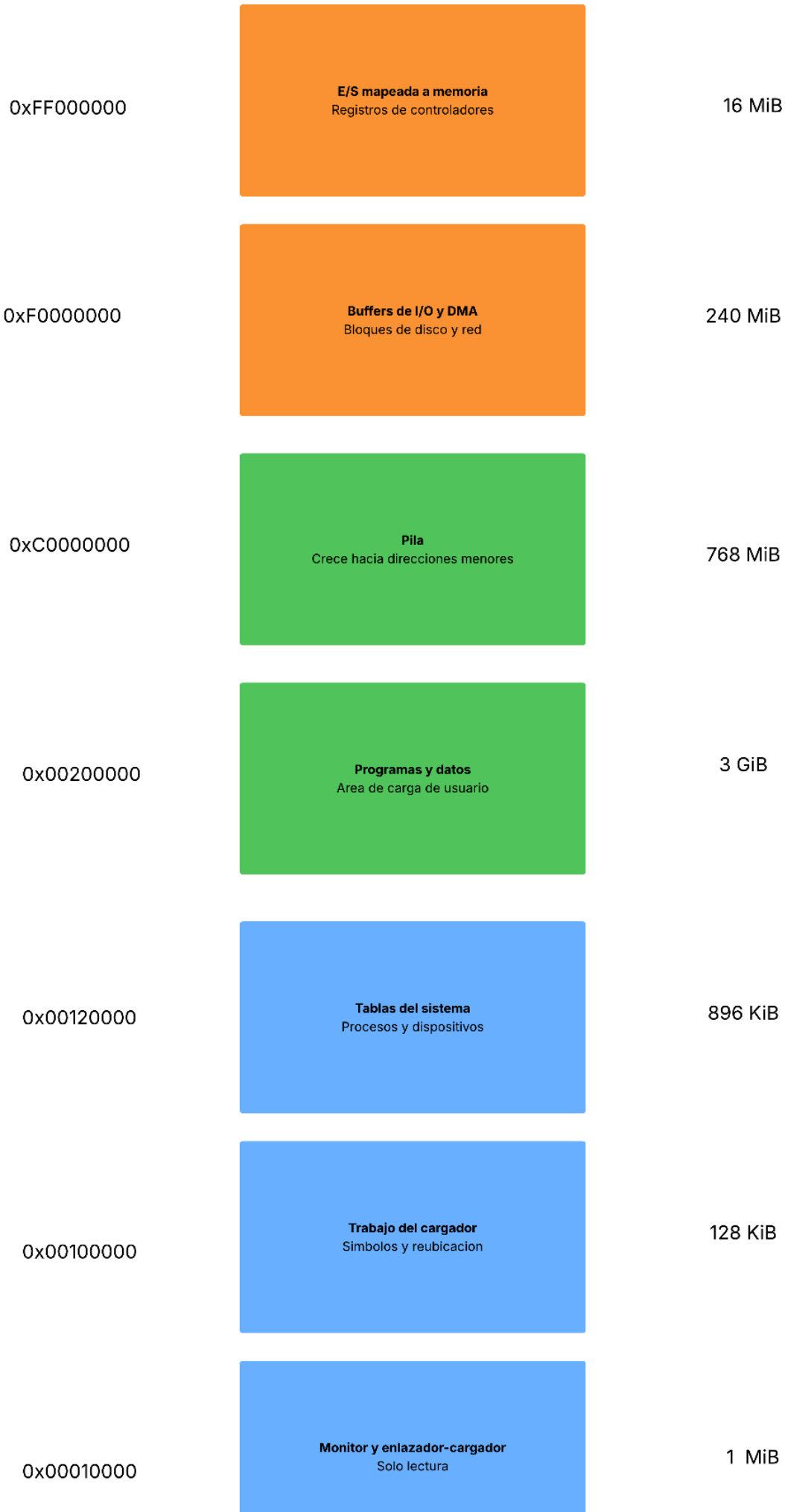
a. *Decodificación de la dirección A[63:0]:*

i. A[2:0] → 3 bits: selecciona el byte dentro de la palabra (para accesos de 1, 2 o 4 bytes).
ii. A[31:3] → 29 bits: selecciona la palabra dentro de la RAM.
iii. A[63:32] → debe ser 0. Si no, la unidad de memoria activa FALLO_DIR en el bus de control (dirección no mapeada).

b. *Tamaño de acceso soportado:*

i. Codificado en 2 bits en el bus de control.
   1. 00 = byte (8b)
   2. 01 = cuarto de palabra (16b)
   3. 10 = media palabra (32b)
   4. 11 = palabra completa (64b)

5. *Mapa de memoria*


0xFF000000
E/S mapeada a memoria
Registros de controladores
16 MiB
0xF0000000
Buffers de I/O y DMA
Bloques de disco y red
240 MiB
0xC0000000
Pila
Crece hacia direcciones menores
768 MiB
0x00200000
Programas y datos
Area de carga de usuario
3 GiB
0x00120000
Tablas del sistema
Procesos y dispositivos
896 KiB
0x00100000
Trabajo del cargador
Simbolos y reubicacion
128 KiB
0x00010000
Monitor y enlazador-cargador
Solo lectura
1 MiB

Sistema
Usuario
I/O—-

Rangos y permisos

Región	Rango completo	Permisos
Vectores	0x00000000 – 0x00000FFF	R / X
Monitor + Enlazador-Cargador	0x00001000 – 0x000FFFFF	R / X
Trabajo del cargador	0x00100000 – 0x0011FFFF	R / W (solo modo admin)
Tablas del sistema	0x00120000 – 0x001FFFFF	R / W (solo modo admin)
Programas y datos	0x00200000 – 0xBFFFFFFF	R / W / X
Pila	0xC0000000 – 0xEFFFFFFF	R / W (no ejecutable)
Buffers de I/O y DMA	0xF0000000 – 0xFEFFFFFF	R / W (no cacheable)
I/O mapeada	0xFF000000 – 0xFFFFFFFF	R / W (no cacheable, admin)
No mapeado	0x0000000100000000 – 0xFFFFFFFFFFFFFFFF	Ninguno → excepción

Cosas a tener en cuenta:

- **La dirección 0x00000000 es el vector de reset.** Al encender, el PC se carga con 0x00000000. Ese es el punto de arranque de toda la máquina.
- **La pila arranca en 0xEFFFFFFF** y crece hacia abajo, mientras el heap del área de usuario crece hacia arriba desde el final de los datos. Quedan separados por 768 MiB, así que nunca chocan en la práctica.

6. I/O Mapeada a memoria

Los controladores exponen sus registros como direcciones de memoria, y el programador los lee y escribe con las mismas instrucciones LOAD y STORE.

Cada controlador recibe una página de 4 KiB:

Dirección base	Controlador
0xFF000000	Entrada (teclado)
0xFF001000	Salida (pantalla)
0xFF002000	Memoria secundaria (disco)
0xFF003000	Interfaz de red
0xFF004000	Temporizador / reloj de tiempo real

Dentro de cada página, los registros van en desplazamientos fijos de 8 bytes:

+0x00 CTRL · +0x08 STATUS · +0x10 DATA · +0x18 ADDR · +0x20 COUNT

Para la interfaz de red, esos campos se especializan: NET_CTRL (iniciar transmisión/recepción), NET_STATUS (listo, error, paquete disponible), NET_MAC (dirección física de 48 bits), NET_TX_ADDR y NET_TX_LEN (puntero y longitud del paquete a enviar, que vive en la región de buffers), NET_RX_ADDR y NET_RX_LEN.

7. Memoria secundaria (Depende del tamaño, en este caso supongamos 1 TiB)

La memoria secundaria no forma parte del espacio de direcciones. Se organiza en bloques y se accede por transferencia DMA a través del controlador de disco.

- Capacidad: 1 TiB = 2^{40} bytes
- Tamaño de bloque: 4 KiB → 2^{28} bloques
- Direccionamiento: LBA (Logical Block Address) de 28 bits, que cabe holgadamente en un registro de 64 bits
- Estructura: bloque 0 = superbloque (metadatos del volumen), bloques 1–1023 = tabla de bloques libres, bloques 1024 en adelante = área de archivos y módulos objeto

Secuencia de lectura de un bloque:

1. Escribir el LBA en DSK_ADDR (0xFF002018)
2. Escribir la dirección de RAM destino y el número de bloques en DSK_COUNT (0xFF002020)
3. Escribir el comando de lectura en DSK_CTRL (0xFF002000)
4. Esperar (por sondeo o interrupción) a que DSK_STATUS indique fin de transferencia

8. Protocolo de acceso de memoria (Interfaz con los buses) → Depende tambien de Sebastian…Entonces el siguiente es solo para guiarnos

Señal	Dirección	Función
MEM_READ	CPU → Memoria	Solicita lectura
MEM_WRITE	CPU → Memoria	Solicita escritura
MEM_SIZE[1:0]	CPU → Memoria	Tamaño del acceso (byte/16/32/64)
READY	Memoria → CPU	Dato válido en el bus de datos
WAIT	Memoria → CPU	Inserta ciclos de espera
FALLO_DIR	Memoria → CPU	Dirección fuera del mapa o permiso denegado

Cosas a tener en cuenta

Ciclo de lectura: la CPU coloca la dirección en el bus de direcciones y activa MEM_READ + MEM_SIZE. La memoria decodifica, activa WAIT mientras accede a la DRAM, coloca la palabra en el bus de datos y activa READY. La CPU toma el dato y libera el bus.

Prebúsqueda de instrucciones (clave por la longitud variable): como una instrucción puede empezar en cualquier byte y cruzar el límite de una palabra, la unidad de memoria siempre entrega la palabra alineada de 64 bits que contiene el byte apuntado por el PC. Un buffer de prebúsqueda en la CPU se encarga de extraer la instrucción y solicitar la siguiente palabra si le faltan bytes.

Especificación Arquitectónica

En este apartado se hace énfasis en la decodificación de las direcciones y el aislamiento de la memoria física con respecto a los periféricos para que el ciclo de la arquitectura se cumpla satisfactoriamente.

1. Decodificación de Memoria y Enrutamiento Físico:

Anteriormente hemos definido la decodificación de la dirección A[63:0], sin embargo, es importante hacer énfasis en la validación del rango lógico y el enrutamiento, por lo cual, la Unidad de Gestión de Memoria implementa la siguiente decodificación por hardware puro antes de activar cualquier línea externa:

- Validación de Rango Lógico (Bits 63-32)

Mediante compuertas combinacionales, se verifica que A[63:32] == 0x00000000. Si algún bit superior es 1, la máquina de estados aborta el ciclo de bus y levanta internamente una excepción FALLO_DIR. Esto evita búsquedas falsas fuera de los 4 GiB soportados.

- Enrutamiento (Bits 31-24)

Si la dirección es válida, el decodificador evalúa el byte más significativo de la parte implementada. Si A[31:24] == 0xFF (es decir, las direcciones que inician en 0xFF000000), se bloquea el acceso a la RAM y se activa la línea de Chip Select del bus de periféricos (MMIO). Para cualquier otro valor, se accede a la memoria RAM estándar (0x00000000 - 0xFEFFFFFF).

2. Modelo de Registros de la CPU:

En este apartado, se especifica cómo se diferencia el uso de los registros arquitectónicos y los microarquitectónicos, es decir los usados para el ISA y los usados para la ruta interna de datos, en nuestro computador, definimos 10 registros para las operaciones ISA y usamos 6 registros para la ruta interna de datos:

- ***Registros Arquitectónicos:***

Registro	Nombre	Propósito y Comportamiento	Estado de Reset
R0	Zero / GPR	Registro cableado a cero para simplificar el ISA (ej. para limpiar registros o hacer comparaciones rápidas). Todo intento de escritura a R0 es ignorado.	0x0000000000000000
R1 - R4	GPRs	Registros de propósito general. Destinados a manipulación de datos, ALU y paso de los primeros 4 argumentos de funciones.	No Definido
R5	RV	*Return Value*. Convención de llamadas para retornar los resultados de las subrutinas.	No Definido
R6	SP (Stack Pointer)	Apunta al tope de la pila en memoria. Como la pila crece hacia abajo, la CPU disminuye el valor de SP *antes* de almacenar datos.	0x00000000EFFFFFFF
R7	BP (Base Pointer)	Mantiene el marco de pila actual (*Stack Frame*) permitiendo el acceso relativo a variables locales y argumentos suplementarios.	No Definido
PC	Program Counter	Almacena la dirección del primer byte de la instrucción a ser "buscada" en memoria. Gestiona la secuencia de ejecución.	0x0000000000000000
SR	Status Register	Registro de estado compuesto. Almacena las banderas de la ALU, control de interrupciones, excepciones y modos de privilegio de hardware.	0x0000000000000001

- ***Registros Micro Arquitectónicos:***

Registro	Ancho	Función en la Arquitectura
MAR	64 bits	**Memory Address Register**. Almacena la dirección física exacta que se va a colocar en el bus de direcciones. Se carga desde el registro PC (en "Búsqueda") o desde el resultado de la ALU (en LOAD/STORE).
MDR	64 bits	**Memory Data Register**. Latch bidireccional conectado al bus de datos. Recibe la palabra de 64 bits completa desde la memoria RAM o los periféricos, o sostiene el dato a escribir temporalmente.
IR	64 bits	**Instruction Register**. Contiene la instrucción en ejecución. Este registro es fundamental para que la lógica combinacional de la Unidad de Control decodifique el Código de Operación y orqueste las señales de control.
A y B	64 bits	**Latches ALU**. Almacenan temporalmente los operandos extraídos del banco de registros. Aíslan la ALU de los cambios del bus interno para prevenir condiciones de carrera o errores de ejecución.
Z	64 bits	**Salida ALU**. Sostiene el resultado de la ALU de forma asíncrona temporalmente antes de devolverlo al banco de registros en la fase de Write-Back, sincronizado con el reloj.

3. ALU:

La Unidad aritmético-Lógica (ALU) se encargará de ejecutar las operaciones centrales de la CPU. En este caso, la idea principal es ejecutar paralelamente las primitivas según las instrucciones decodificadas provenientes de la Unidad de Control:

- **Aritméticas:** ADD (suma), SUB (resta), MUL (multiplicación), DIV (división), Incluyendo la lógica de detección de Carry y Overflow.
- **Lógicas:** AND, OR, XOR, NOT bit a bit.
- **Desplazamientos:** SHL (izq. lógico), SHR (der. lógico), ASR (der. aritmético).

- ***Banderas de Estado (Registro SR):***

Estas banderas se actualizarán automáticamente luego de una instrucción aritmética/lógica o de acceso a memoria:

- **[0] Z (Zero):** Activada a 1 si el resultado exacto de la operación de la ALU es 0 en todos sus 64 bits.
- **[1] N (Negative):** Refleja el estado directo del bit 63 del resultado (MSB). Determina valores en Complemento a 2 negativos.
- **[2] C (Carry):** Acarreo más allá del bit 63 (operaciones sin signo).
- **[3] V (Overflow):** Desbordamiento con signo. Ocurre si al sumar dos positivos el resultado da negativo, o viceversa.
- **[4] M (Misaligned):** Error de alineación de memoria. Levantado por la fase MEMORY de la máquina de estados.
- **[5] I (Interrupt):** Bit habilitador. Si es 1, la CPU interrumpe su FSM (Máquina de Estado Finito) después del Write-Back para atender al Controlador DMA / Temporizador.
- **[6] S (Supervisor):** Nivel de anillo de ejecución (1: Admin/Kernel, 0: Usuario). Previene accesos a rangos críticos del mapa de memoria.

4. Unidad de Control:

La CU está organizada como una FSM multiciclo a nivel de Hardware. Como las instrucciones son de longitud variable, agregamos un Buffer de Pre-fetch (Alistamiento de búsqueda) de 16 bytes para mitigar el efecto de las instrucciones no alineadas a 64 bits.

- ***FETCH:*** El valor del registro PC se envía al MAR. Aquí, la memoria extrae y envía una palabra completa de 64 bits al MDR, luego la lógica de control ubica esos bytes en la cola del Buffer del Pre-Fetch. Luego el código de operación se transfiere al registro IR y el PC se autoincrementa según el conteo exacto de bytes detectado del código de operación actual.
- ***DECODE:*** Aquí,el Arreglo Lógico Programable (PLA) de la CU desglosa los bits del registro IR, en este paso se evaluarán los privilegios comprobando el registro SR frente a la dirección solicitada. Los operandos seleccionados del banco se cargan en los latches A y B.
- ***EXECUTE:*** La ALU realiza la operación sobre A y B, luego el resultado se almacena temporalmente en Z y se calculan las banderas Z, N, C, V para el volcado al SR. En caso de que la instrucción sea de salto, la ALU simplemente computa la dirección de destino sumando el desplazamiento al registro PC.
- ***MEMORY:*** Este ciclo se usa en el LOAD/STORE. Aquí el Registro Z funciona como puntero y se mueve al MAR. Se hace la verificación sobre A[62-32] ==0 y se comprueba la alineación (Pre-fetch). En el STORE la CPU ubica el registro general original en el MDR y activa la señal de escritura al bus y en el LOAD la memoria escribe la respuesta en el MDR.

- ***WRITE BACK:*** El resultado final (en Z o en la MDR) se envía por los buses internos para reescribir permanentemente los registros de destino (del 1 al 7). Luego se verifica si hay señales del DMA del disco o E/S para llevar el volcado de contexto a la pila si es necesario.

Diseño del Conjunto de Instrucciones (ISA)

En este apartado se define la Arquitectura del Conjunto de Instrucciones (ISA) para nuestra máquina, abarcando la estructura de los formatos de instrucción y el grupo de microinstrucciones base correspondientes a operaciones aritméticas, lógicas y de acceso a memoria.

1. Formatos de Instrucción de Longitud Variable:

Dado que nuestra arquitectura exige instrucciones de longitud variable alineadas a byte y contamos con 8 registros arquitectónicos (R0 a R7), hemos decidido utilizar 4 bits (un nibble) para codificar cada registro. Esto mantiene las instrucciones compactas y permite que la traducción manual a código de máquina (hexadecimal) sea directa. El código de operación (Opcode) siempre ocupará el primer byte (8 bits).
Se definen los siguientes formatos base para las instrucciones:

- Formato 1: Registro a Registro (3 bytes)
  Utilizado para operaciones aritméticas y lógicas de la ALU que requieren tres registros (un destino y dos orígenes).
  - **Byte 0:** Opcode (8 bits)
  - **Byte 1:** Registro Destino `Rd` (4 bits) | Registro Origen 1 `Rs1` (4 bits)
  - **Byte 2:** Registro Origen 2 `Rs2` (4 bits) | No usado `0000` (4 bits)
- Formato 2: Un Operando (2 bytes)
  Utilizado para operaciones unarias (ej. NOT, incrementos).
  - **Byte 0:** Opcode (8 bits)
  - **Byte 1:** Registro Destino/Origen `Rd` (4 bits) | No usado `0000` (4 bits)
- Formato 3: Inmediato y Memoria (4 bytes)
  Utilizado para aritméticas con constantes (inmediatos) y accesos a memoria RAM (LOAD/STORE).
  - **Byte 0:** Opcode (8 bits)
  - **Byte 1:** Registro Destino/Origen `Rd` o `Rs` (4 bits) | Registro Base `Rb` o `Rs1` (4 bits)
  - **Bytes 2 y 3:** Valor Inmediato o Desplazamiento (*Offset*) (16 bits)

- Formato 4: Salto relativo
  Utilizado para instrucciones de salto condicional que modifican el PC mediante un desplazamiento relativo de 16 bits. El desplazamiento se interpreta con signo respecto al PC que apunta a la siguiente instrucción.
  - **Byte 0**: Opcode (8 bits)
  - **Bytes 1 y 2**: Desplazamiento relativo Offset 16 (16 bits con signo)

- Formato 5: Dirección Absoluta (5 bytes)
  Utilizado para instrucciones que requieren especificar directamente una dirección de memoria de 32 bits, como saltos incondicionales y llamadas a subrutinas.
  - **Byte 0:** Opcode (8 bits)
  - **Bytes 1, 2, 3 y 4**: Dirección absoluta (32 bits)
- Formato 6: Registro (2 bytes)
  Utilizado para instrucciones que requieren un único registro como operando, incluyendo operaciones de salto indirecto, llamadas a subrutinas mediante registros y operaciones de pila.
  - **Byte 0**: Opcode (8 bits)
  - **Byte 1**: Registro Rn (4 bits) | No usado 0000 (4 bits)

2. Codificación de los Registros

La arquitectura dispone de 10 registros arquitectónicos para el ISA, conformados por cinco registros de propósito general identificados desde R0 hasta R4, y los registros de propósito específico de R5 a R7,además dos registros especiales: PC (Program Counter) y SR (Status Register). Cada registro se codifica mediante 4 bits (un nibble) dentro de los formatos de instrucción, permitiendo representar hasta 16 registros. Los códigos restantes se reservan para futuras extensiones de la arquitectura.

Registro	Código (4 bits)	Función
R0	0000	Registro constante en cero
R1	0001	Propósito general
R2	0010	Propósito general
R3	0011	Propósito general
R4	0100	Propósito general
R5	0101	Registro de valor de retorno (RV)
R6	0110	Stack Pointer (SP)
R7	0111	Base Pointer (BP)
PC	1000	Program Counter; dirección de la siguiente instrucción
SR	1001	Status register; contiene las banderas y bits de control
R10	1010	Reservado
R11	1011	Reservado
R12	1100	Reservado
R13	1101	Reservado
R14	1110	Reservado
R15	1111	Reservado

3. *Repertorio de Microinstrucciones Base*

Estas instrucciones cubren las necesidades fundamentales de la ALU y la interacción con la memoria. Las banderas del registro SR (Z, N, C, V, M) se actualizan automáticamente según lo especificado en el diseño de la CPU en la sección anterior.

- *Aritméticas*

Mnemónico	**Opcode**	**Formato**	**Acción operacional de registros**	**Explicación**	**Banderas**
ADD	0x10	3 bytes	Rd ← Rs1 + Rs2	Suma el contenido de dos registros y almacena el resultado en el registro destino.	Z, N, C, V
SUB	0x11	3 bytes	Rd ← Rs1 - Rs2	Resta el segundo registro del primero y guarda el resultado en el registro destino.	Z, N, C, V
MUL	0x12	3 bytes	Rd ← Rs1 * Rs2	Multiplica dos registros de 64 bits y almacena los 64 bits menos significativos.	Z, N

DIV	0x13	3 bytes	Rd ← Rs1 / Rs2	Realiza la división entera entre dos registros y guarda el cociente en destino.	Z, N
ADDI	0x14	4 bytes	Rd ← Rs1 + SignExtend(Imm16)	Suma un valor inmediato de 16 bits con extensión de signo a un registro.	Z, N, C, V
SUBI	0x15	4 bytes	Rd ← Rs1 - SignExtend(Imm16)	Resta un inmediato de 16 bits con extensión de signo al registro fuente.	Z, N, C, V
INC	0x16	2 bytes	Rd ← Rd + 1	Incrementa en uno el valor almacenado en el registro especificado.	Z, N, V

DEC	0x17	2 bytes	Rd ← Rd - 1	Decrementa en uno el valor almacenado en el registro especificado.	Z, N, V

- ***Lógicas y Desplazamientos***

Mnemónico	Opcode	Formato	Acción operacional de registros	Explicación	Banderas
AND	0x20	3 bytes	Rd ← Rs1 & Rs2	Realiza la conjunción lógica bit a bit entre dos registros.	Z, N
OR	0x21	3 bytes	Rd ← Rs1 | Rs2	Realiza la disyunción lógica bit a bit entre dos registros.	Z, N
XOR	0x22	3 bytes	Rd ← Rs1 ^ Rs2	Aplica una operación lógica XOR bit a bit entre dos registros.	Z, N

NOT	0x23	2 bytes	Rd ← ~Rd	Invierte todos los bits del registro destino (complemento a 1).	Z, N
SHL	0x24	4 bytes	Rd ← Rs1 << Imm16	Desplaza los bits de un registro a la izquierda rellenando con ceros.	Z, N, C
SHR	0x25	4 bytes	Rd ← Rs1 >> Imm16	Desplaza los bits de un registro a la derecha rellenando con ceros.	Z, N, C
ASR	0x26	4 bytes	Rd ← Rs1 >> Imm16 (Aritmético)	Desplaza los bits a la derecha preservando el bit de signo (MSB).	Z, N, C
CMP	0x27	3 bytes	Temp ← Rs1 - Rs2	Resta dos registros actualizando las banderas sin modificar los registros.	Z, N, C, V

- ***Carga y Almacenamiento con Memoria***

Para los accesos a memoria, la Unidad de Control evalúa el registro base (Rb) sumado al desplazamiento de 16 bits para calcular la dirección física que se enviará al MAR. Si la dirección de 64 bits no está alineada, se levantará la bandera M (Misaligned) detallada previamente.

Mnemónico	**Opcode**	**Formato**	**Acción operacional de registros**	**Explicación**	**Banderas**
LOAD	0x30	4 bytes	Rd ← Memoria[Rb + SignExtend(Offset16)]	Carga una palabra de 64 bits desde la memoria RAM hacia un registro.	M
STORE	0x31	4 bytes	Memoria[Rb + SignExtend(Offset16)] ← Rs	Escribe una palabra completa de 64 bits desde un registro hacia la memoria.	M
LDB	0x32	4 bytes	Rd ← ZeroExtend(Memoria[Rb + SignExtend(Offset16)])	Carga un byte desde memoria rellenando con ceros los 56 bits superiores.	Ninguna

STB	0x33	4 bytes	Memoria[Rb + SignExtend(Offset16)] ← Rs[7:0]	Escribe el byte menos significativo de un registro en la dirección de memoria.	Ninguna

- Control de Flujo y Saltos

Las instrucciones de control de flujo permiten modificar la secuencia normal de ejecución del programa mediante cambios en el registro PC. Los saltos condicionales utilizan las banderas Z, N, C y V almacenadas en el registro SR, previamente actualizadas por las operaciones de la ALU, mientras que los saltos incondicionales permiten especificar directamente una nueva dirección de ejecución.

Mnemónico	Opcode	Formato	Acción operacional de registros	Explicación	Banderas
JMP	0x40	5 bytes	PC ← ZeroExtend(Addr32)	Carga directamente en el PC una dirección absoluta de 32 bits.	Ninguna
JZ	0x41	3 bytes	Si SR.Z = 1: PC ← PC + SignExtend(Offset16)	Realiza un salto relativo si la bandera de cero está activa.	Ninguna
JNZ	0x42	3 bytes	Si SR.Z = 0: PC ← PC + SignExtend(Offset16)	Realiza un salto relativo cuando el resultado anterior no fue cero.	Ninguna
JC	0x43	3 bytes	Si SR.C = 1: PC ← PC + SignExtend(Offset16)	Salta si existe acarreo en la operación anterior.	Ninguna
JNC	0x44	3 bytes	Si SR.C = 0: PC ← PC + SignExtend(Offset16)	Salta cuando no existe acarreo.	Ninguna

JN	0x45	3 bytes	Si SR.N = 1: PC ← PC + SignExtend(Offset16)	Salta cuando el resultado anterior tiene signo negativo.	Ninguna
JP	0x46	3 bytes	Si SR.N = 0 ∧ SR.Z = 0: PC ← PC + SignExtend(Offset16)	Salta cuando el resultado anterior es positivo.	Ninguna
JV	0x47	3 bytes	Si SR.V = 1: PC ← PC + SignExtend(Offset16)	Salta si se produjo un desbordamiento con signo.	Ninguna
JNV	0x48	3 bytes	Si SR.V = 0: PC ← PC + SignExtend(Offset16)	Salta cuando no se produjo desbordamiento con signo.	Ninguna
JMPR	0x49	2 bytes	PC ← Rn	Carga el PC con la dirección contenida en un registro.	Ninguna

- ***Subrutinas***

Las instrucciones de subrutinas permiten implementar llamadas y retornos de funciones. Para almacenar temporalmente la dirección de retorno se utiliza la pila mediante el registro SP (R6). Las instrucciones CALL y CALLR almacenan la dirección de la siguiente instrucción antes de transferir el control a la subrutina, mientras que RET recupera dicha dirección para continuar la ejecución del programa.

Mnemónico	**Opcode**	**Formato**	**Acción operacional de registros**	**Explicación**	**Banderas**
CALL	0x4A	5 bytes	SP ← SP - 8 MAR ← SP MDR ← PC Mem[MAR] ← MDR PC←ZeroExtend(Addr32)	Guarda en la pila la dirección de retorno y posteriormente salta a la dirección absoluta de la subrutina.	Ninguna

CALLR	0x4B	2 bytes	SP ← SP - 8 MAR ← SP MDR ← PC Mem[MAR] ← MDR PC ← Rn	Guarda la dirección de retorno en la pila y salta a la dirección contenida en un registro.	Ninguna
RET	0x4C	1 byte	MAR ← SP MDR ← Mem[MAR] PC ← MDR SP ← SP + 8	Recupera de la pila la dirección de retorno y continúa la ejecución desde ella.	Ninguna

- *Manejo de la Pila*

Las instrucciones de manejo de pila permiten almacenar y recuperar temporalmente información utilizando el registro SP (R6). Debido a que la arquitectura trabaja con datos de 64 bits, cada operación de pila modifica el SP en 8 bytes. Además, ENTER y LEAVE permiten establecer y liberar el marco de pila de una subrutina mediante el registro BP (R7).

Mnemónico	Opcode	Formato	Acción operacional de registros	Explicación	Banderas
PUSH	0x4D	2 bytes	SP ← SP - 8 MAR ← SP MDR ← Rn Mem[MAR] ← MDR	Reserva una palabra de 64 bits en la pila y almacena allí el contenido del registro.	Ninguna
POP	0x4E	2 bytes	MAR ← SP MDR ← Mem[MAR] Rn ← MDR SP ← SP + 8	Recupera la palabra ubicada en el tope de la pila y aumenta el SP.	Ninguna
ENTER	0x4F	4 bytes	SP ← SP - 8 MAR ← SP MDR ← BP Mem[MAR] ← MDR BP ← SP SP ← SP - Imm16	Crea un nuevo marco de pila, guarda el BP anterior y reserva espacio para variables locales.	Ninguna

LEAVE	0x50	1 byte	SP ← BP MAR ← SP MDR ← Mem[MAR] BP ← MDR SP ← SP + 8	Libera el marco actual de la pila y restaura el BP anterior.	Ninguna

- *Control de Interrupciones*

Las instrucciones de control de interrupciones permiten habilitar o deshabilitar temporalmente la atención de interrupciones y retornar desde una rutina de servicio. Estas operaciones utilizan el bit I del registro de estado SR. La instrucción IRET permite restaurar el contexto de ejecución almacenado en la pila.

Mnemónico	Opcode	Formato	Acción operacional de registros	Explicación	Banderas
EI	0x51	1 byte	SR.I ← 1	Habilita la atención de interrupciones mediante el bit I del registro de estado.	I
DI	0x52	1 byte	SR.I ← 0	Deshabilita temporalmente la atención de interrupciones.	I
IRET	0x53	1 byte	MAR ← SP MDR ← Mem[MAR] PC ← MDR SP ← SP + 8 MAR ← SP MDR ← Mem[MAR] SR ← MDR SP ← SP + 8	Recupera de la pila el PC y el SR almacenados al entrar en una rutina de interrupción.	Ninguna
HLT	0x00	1 byte	CU.HALT ←1	Detiene la ejecución del procesador. La Unidad de Control deja de iniciar nuevos ciclos de FETCH y el procesador permanece detenido hasta recibir un RESET.	Ninguna

Módulo Enlazador-Cargador. Tareas del Cargador.

1. Propósito y alcance

El Cargador es el componente del módulo Enlazador-Cargador responsable de tomar un programa ya traducido a lenguaje de máquina y ubicarlo en la memoria RAM a partir de una dirección dada, dejando el procesador listo para iniciar su ejecución desde allí. No traduce ensamblador ni resuelve símbolos entre módulos (tarea del Enlazador); su función es la ubicación física en memoria y la transferencia de control al programa cargado.

2. Relación con el resto de la arquitectura

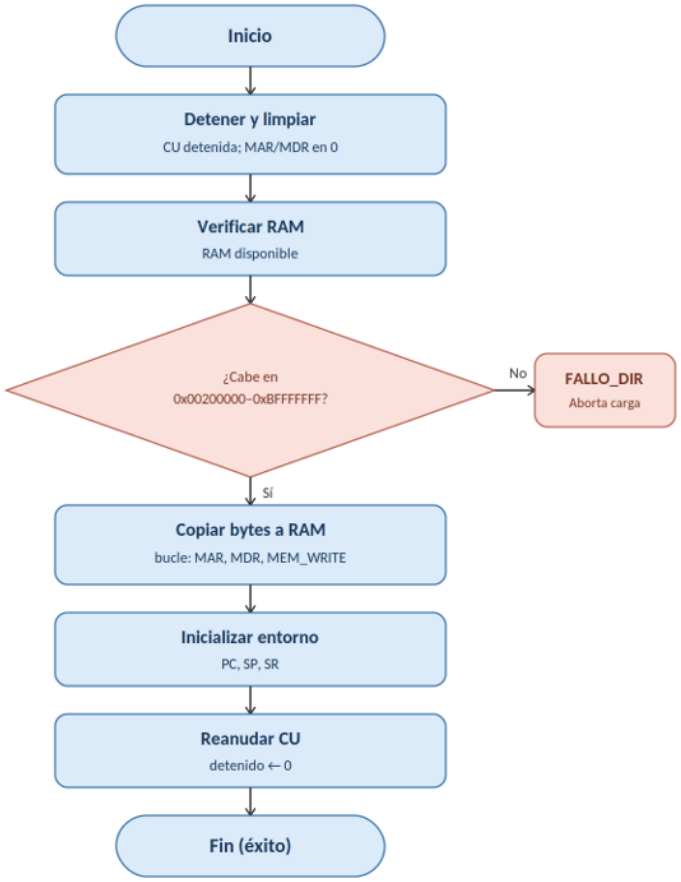
El mapa de memoria del asigna al bloque "Monitor y enlazador-cargador" (0x00001000–0x000FFFFF) permisos **R/X** — es decir, es memoria de código ejecutable, igual que cualquier otro programa. Por eso el Cargador se diseña **como un programa en el mismo ensamblador de la máquina**, no como un bloque de hardware aparte: reutiliza la ALU, la Unidad de Control y las instrucciones LDB/STB ya definidas, sin necesitar mecanismos nuevos. Esto es consistente con la filosofía von Neumann del propio proyecto: programa y datos (incluido el programa que carga otros programas) viven en la misma memoria.

3. Convención de invocación

Registro	Contenido al invocar CARGADOR
R1	ORIGEN — dirección donde está el código ya traducido (p. ej., tras leerse de disco)
R2	DESTINO — direccion_base, dónde debe quedar el programa
R3	TAMAÑO — cantidad de bytes a copiar

4. Algoritmo conceptual

1. **Validación**: DESTINO debe estar dentro de la región "Programas y datos" (0x00200000–0xBFFFFFFF), la única con permisos R/W/X para código de usuario. Se descarta cualquier destino que invada zonas de sistema o la pila.
2. **Copia**: se recorre el binario byte a byte desde ORIGEN y se escribe en DESTINO, usando las instrucciones de acceso a memoria de un byte (LDB/STB) — la fase MEMORY de la FSM ya definida por el grupo se encarga de MAR/MDR sin necesidad de nada adicional.
3. **Transferencia de control**: al terminar la copia, se salta a la dirección de destino (JMPR), arrancando el programa recién cargado.



5. Implementación en ensamblador

None

```
CARGADOR:
    ; --- Validación 1: DESTINO >= 0x00200000 ---
    ADDI R4, R0, 0x0020
    SHL  R4, R4, 16          ; R4 = 0x00200000
    CMP  R2, R4
    JN   CARGADOR_ERROR      ; si DESTINO < 0x00200000 → error

    ; --- Validación 2: DESTINO + TAMAÑO <= 0xC0000000 (no
invadir la Pila) ---
    ADD  R5, R2, R3          ; R5 = destino final (temporal)
    ADDI R4, R0, 3
    SHL  R4, R4, 30          ; R4 = 0xC0000000
    CMP  R5, R4
    JP   CARGADOR_ERROR      ; si excede el límite → error
```

```
    ADDI R5, R2, 0           ; R5 = direccion_base (para el salto
final)

CARGADOR_COPIA:
    CMP  R3, R0
    JZ   CARGADOR_FIN
    LDB  R4, [R1 + 0]
    STB  R4, [R2 + 0]
    ADDI R1, R1, 1
    ADDI R2, R2, 1
    SUBI R3, R3, 1
    JMP  CARGADOR_COPIA

CARGADOR_FIN:
    JMPR R5                  ; PC ← direccion_base: arranca el
programa cargado

CARGADOR_ERROR:
    HLT
```

Nota de diseño: R5 se reutiliza para dos propósitos secuenciales (acumulador temporal en la validación, luego portador de direccion_base) porque el primer uso ya no se necesita cuando comienza el segundo. Deliberadamente no se usan R6 (SP) ni R7 (BP) como registros de trabajo, para no interferir con la pila del sistema.

6. Traducción manual a lenguaje de máquina (Big-Endian), cargado en 0x00001000

Dirección	Instrucción	Formato	Código de Máquina (Hex)
0x00001000	ADDI R4, R0, 0x0020	F3 (4B)	14 40 00 20
0x00001004	SHL R4, R4, 16	F3 (4B)	24 44 00 10
0x00001008	CMP R2, R4	F1 (3B)	27 02 40
0x0000100B	JN CARGADOR_ERROR	F4 (3B)	45 00 36
0x0000100E	ADD R5, R2, R3	F1 (3B)	10 52 30

0x00001011	ADDI R4, R0, 3	F3 (4B)	14 40 00 03
0x00001015	SHL R4, R4, 30	F3 (4B)	24 44 00 1E
0x00001019	CMP R5, R4	F1 (3B)	27 05 40
0x0000101C	JP CARGADOR_ERROR	F4 (3B)	46 00 25
0x0000101F	ADDI R5, R2, 0	F3 (4B)	14 52 00 00
0x00001023 CARGADOR_COPIA:	CMP R3, R0	F1 (3B)	27 03 00
0x00001026	JZ CARGADOR_FIN	F4 (3B)	41 00 19
0x00001029	LDB R4, [R1+0]	F3 (4B)	32 41 00 00
0x0000102D	STB R4, [R2+0]	F3 (4B)	33 42 00 00
0x00001031	ADDI R1, R1, 1	F3 (4B)	14 11 00 01
0x00001035	ADDI R2, R2, 1	F3 (4B)	14 22 00 01
0x00001039	SUBI R3, R3, 1	F3 (4B)	15 33 00 01
0x0000103D	JMP CARGADOR_COPIA	F5 (5B)	40 00 00 10 23
0x00001042 CARGADOR_FIN:	JMPR R5	F6 (2B)	49 50
0x00001044 CARGADOR_ERROR:	HLT	1B	00

Los desplazamientos de los saltos relativos se calculan respecto a la dirección de la instrucción siguiente al salto, igual que en los tres algoritmos ya documentados (p. ej. JN en 0x0000100B salta a 0x00001044: 0x1044 − 0x100E = 0x36).

7. Manejo de errores

Condición	Detección	Efecto
DESTINO < 0x00200000	CMP R2,R4 + JN	Salta a CARGADOR_ERROR, ejecuta HLT sin copiar nada
DESTINO + TAMAÑO > 0xC0000000	CMP R5,R4 + JP	Salta a CARGADOR_ERROR, ejecuta HLT sin copiar nada

8. Verificación

Invocando CARGADOR con R1=0x00050000 (donde supuestamente llegó el binario del Algoritmo 1 — factorial, 41 bytes — tras leerse de disco), R2=0x00200000, R3=41:

- Validación 1: 0x00200000 − 0x00200000 = 0 → N=0 → no salta a error
- Validación 2: destino final 0x00200029, comparado con 0xC0000000 → muy por debajo → N=1 → no salta a error
- Se copian los 41 bytes de 0x00050000 a 0x00200000
- JMPR R5 con R5=0x00200000 → PC=0x00200000 → el primer FETCH lee 14 10 00 20 → ADDI R1,R0,0x0020, exactamente el inicio del factorial ya verificado.

Implementación y Verificación de Algoritmos

1. Algoritmo 1: Cálculo del Factorial (N!)

Descripción y Pseudocódigo

Calcula el factorial de un número natural N ≥ 0 cargado desde la memoria RAM.

```
HTML
// Variables en memoria RAM (Dirección base: 0x00201000)
// N = 5
N = Mem[0x00201000]
factorial = 1
si N == 0 entonces:
    ir a FIN_FACT
fin si

mientras N > 0 hacer:
    factorial = factorial * N
    N = N - 1
fin mientras

FIN_FACT:
Mem[0x00201008] = factorial
// Resultado esperado: 5! = 120 (0x0000000000000078)
```

Programa en Lenguaje Ensamblador

- **Dirección base de carga del código:** 0x00200000 (Segmento de usuario)
- **Ubicación de variables en RAM:** 0x00201000
- **Asignación de registros:**
    - **R1:** Puntero base de variables en memoria (0x00201000)
    - **R2:** Valor de entrada N / Contador decremental
    - **R3:** Acumulador del producto (factorial)

```
HTML
;
================================================================
========
; ALGORITMO 1: FACTORIAL DE UN NÚMERO
```

```
;
================================================================
========
0x00200000:  ADDI  R1, R0, 0x0020     ; R1 = 0x0020
0x00200004:  SHL   R1, R1, 16         ; R1 = 0x00200000
0x00200008:  ADDI  R1, R1, 0x1000     ; R1 = 0x00201000 (Base de
datos)
0x0020000C:  LOAD  R2, [R1 + 0]       ; R2 = N (cargado desde
Mem[0x00201000])
0x00200010:  ADDI  R3, R0, 1          ; R3 = factorial = 1
0x00200014:  CMP   R2, R0             ; Compara N con 0
(actualiza Z)
0x00200017:  JZ    FIN_FACT           ; Si N == 0, salta al final
(salto +10B)

LOOP_FACT:
0x0020001A:  MUL   R3, R3, R2         ; factorial = factorial * N
0x0020001D:  SUBI  R2, R2, 1          ; N = N - 1 (actualiza flag
Z)
0x00200021:  JNZ   LOOP_FACT          ; Si N != 0, repetir bucle
(salto -10B)

FIN_FACT:
0x00200024:  STORE R3, [R1 + 8]       ; Mem[0x00201008] =
factorial final
0x00200028:  HLT                      ; Detener ejecución
```

Traducción Manual a Lenguaje de Máquina (Big-Endian)

Dirección	Instrucción Ensamblador	Formato	Desglose Hexadecimal (Big-Endian)	Código de Máquina (Hex)
0x00200000	ADDI R1, R0, 0x0020	F3 (4B)	Op:14 | Rd:1 Rs1:0 | Imm:0020	14 10 00 20

Dirección	Instrucción Ensamblador	Formato	Desglose Hexadecimal (Big-Endian)	Código de Máquina (Hex)
0x00200004	SHL R1, R1, 16	F3 (4B)	Op:24 | Rd:1 Rs1:1 | Imm:0010	24 11 00 10
0x00200008	ADDI R1, R1, 0x1000	F3 (4B)	Op:14 | Rd:1 Rs1:1 | Imm:1000	14 11 10 00
0x0020000C	LOAD R2, [R1 + 0]	F3 (4B)	Op:30 | Rd:2 Rb:1 | Offset:0000	30 21 00 00
0x00200010	ADDI R3, R0, 1	F3 (4B)	Op:14 | Rd:3 Rs1:0 | Imm:0001	14 30 00 01
0x00200014	CMP R2, R0	F1 (3B)	Op:27 | R0:0 Rs1:2 | Rs2:0 0	27 02 00
0x00200017	JZ FIN_FACT	F4 (3B)	Op:41 | Off:0x0024 - 0x001A = +10	41 00 0A
0x0020001A	MUL R3, R3, R2	F1 (3B)	Op:12 | Rd:3 Rs1:3 | Rs2:2 0	12 33 20
0x0020001D	SUBI R2, R2, 1	F3 (4B)	Op:15 | Rd:2 Rs1:2 | Imm:0001	15 22 00 01

Dirección	Instrucción Ensamblador	Formato	Desglose Hexadecimal (Big-Endian)	Código de Máquina (Hex)
0x00200021	JNZ LOOP_FACT	F4 (3B)	Op:42 | Off:0x001A - 0x0024 = -10	42 FF F6
0x00200024	STORE R3, [R1 + 8]	F3 (4B)	Op:31 | Rs:3 Rb:1 | Offset:0008	31 31 00 08
0x00200028	HLT	1 Byte	Op:00	00

Trazado de Verificación Técnica

- **Estado inicial en RAM (0x00201000):** N = 0x0000000000000005 (5 decimal).
- **Configuración inicial:** R1 = 0x00201000, R2 = 5, R3 = 1. CMP R2, R0 genera 5 - 0 = 5 ≠ 0 → Z = 0. JZ no salta.
- **Iteración 1:** R3 = 1 * 5 = 5. SUBI R2, R2, 1 deja R2 = 4 (Z = 0). JNZ bifurca a 0x0020001A.
- **Iteración 2:** R3 = 5 * 4 = 20. SUBI R2, R2, 1 deja R2 = 3 (Z = 0). JNZ bifurca a 0x0020001A.
- **Iteración 3:** R3 = 20 * 3 = 60. SUBI R2, R2, 1 deja R2 = 2 (Z = 0). JNZ bifurca a 0x0020001A.
- **Iteración 4:** R3 = 60 * 2 = 120. SUBI R2, R2, 1 deja R2 = 1 (Z = 0). JNZ bifurca a 0x0020001A.
- **Iteración 5:** R3 = 120 * 1 = 120. SUBI R2, R2, 1 deja R2 = 0 (Z = 1 en SR). JNZ evalúa Z = 1 y no salta.
- Cae en 0x00200024: STORE escribe 120 (0x0000000000000078) en la dirección 0x00201008.
- Se ejecuta HLT en 0x00200028. Resultado matemático verificado con precisión.

2. Algoritmo 2: Algoritmo de Euclides (Máximo Común Divisor)

Descripción y Pseudocódigo

Calcula el máximo común divisor (MCD) de dos números naturales positivos *A* y *B* alojados en RAM mediante el método de restas sucesivas de Euclides.

```
// Variables en memoria RAM (Dirección base: 0x00202000)
// A = 48, B = 18
A = Mem[0x00202000]
B = Mem[0x00202008]

mientras A != B hacer:
    si A < B entonces:
        B = B - A
    sino:
        A = A - B
    fin si
fin mientras

Mem[0x00202010] = A
// Resultado esperado: MCD(48, 18) = 6 (0x0000000000000006)
```

Programa en Lenguaje Ensamblador

- **Dirección base de carga del código:** 0x00200100
- **Ubicación de variables en RAM:** 0x00202000
- **Asignación de registros:**
  - **R1:** Puntero base de variables (0x00202000)
  - **R2:** Variable *A*
  - **R3:** Variable *B*

```
;
================================================================
========
; ALGORITMO 2: ALGORITMO DE EUCLIDES
;
================================================================
========
0x00200100:  ADDI  R1, R0, 0x0020     ; R1 = 0x0020
0x00200104:  SHL   R1, R1, 16         ; R1 = 0x00200000
0x00200108:  ADDI  R1, R1, 0x2000     ; R1 = 0x00202000 (Base de
datos)
0x0020010C:  LOAD  R2, [R1 + 0]       ; R2 = A (48)
0x00200110:  LOAD  R3, [R1 + 8]       ; R3 = B (18)
```

```
LOOP_EUCLIDES:
0x00200114:  CMP   R2, R3             ; Evalúa R2 - R3 (actualiza
flags Z, N)
0x00200117:  JZ    FIN_EUCLIDES       ; Si A == B, proceso
terminado (salto +19B)
0x0020011A:  JN    B_ES_MAYOR         ; Si A < B (N=1), saltar a
restar B (salto +8B)

; Caso A > B:
0x0020011D:  SUB   R2, R2, R3         ; A = A - B
0x00200120:  JMP   0x00200114         ; Salto incondicional al
inicio del bucle

B_ES_MAYOR:
0x00200125:  SUB   R3, R3, R2         ; B = B - A
0x00200128:  JMP   0x00200114         ; Salto incondicional al
inicio del bucle

FIN_EUCLIDES:
0x0020012D:  STORE R2, [R1 + 16]      ; Mem[0x00202010] = MCD
resultante
0x00200131:  HLT                      ; Detener procesador
```

Traducción Manual a Lenguaje de Máquina (Big-Endian)

Formato 5 (5B): [Opcode: 0x40][Dirección Absoluta 32 bits (Big-Endian)]

Dirección	Instrucción Ensamblador	Formato	Desglose Hexadecimal (Big-Endian)	Código de Máquina (Hex)
0x00200100	ADDI R1, R0, 0x0020	F3 (4B)	Op:14 | Rd:1 Rs1:0 | Imm:0020	14 10 00 20
0x00200104	SHL R1, R1, 16	F3 (4B)	Op:24 | Rd:1 Rs1:1 | Imm:0010	24 11 00 10

Dirección	Instrucción Ensamblador	Formato	Desglose Hexadecimal (Big-Endian)	Código de Máquina (Hex)
0x00200108	ADDI R1, R1, 0x2000	F3 (4B)	Op:14 | Rd:1 Rs1:1 | Imm:2000	14 11 20 00
0x0020010C	LOAD R2, [R1 + 0]	F3 (4B)	Op:30 | Rd:2 Rb:1 | Offset:0000	30 21 00 00
0x00200110	LOAD R3, [R1 + 8]	F3 (4B)	Op:30 | Rd:3 Rb:1 | Offset:0008	30 31 00 08
0x00200114	CMP R2, R3	F1 (3B)	Op:27 | R0:0 Rs1:2 | Rs2:3 0	27 02 30
0x00200117	JZ FIN_EUCLIDES	F4 (3B)	Op:41 | Off:0x012D - 0x011A = +19	41 00 13
0x0020011A	JN B_ES_MAYOR	F4 (3B)	Op:45 | Off:0x0125 - 0x011D = +8	45 00 08
0x0020011D	SUB R2, R2, R3	F1 (3B)	Op:11 | Rd:2 Rs1:2 | Rs2:3 0	11 22 30
0x00200120	JMP 0x00200114	F5 (5B)	Op:40 | Addr:00200114	40 00 20 01 14

Dirección	Instrucción Ensamblador	Formato	Desglose Hexadecimal (Big-Endian)	Código de Máquina (Hex)
0x00200125	SUB R3, R3, R2	F1 (3B)	Op:11 | Rd:3 Rs1:3 | Rs2:2 0	11 33 20
0x00200128	JMP 0x00200114	F5 (5B)	Op:40 | Addr:00200114	40 00 20 01 14
0x0020012D	STORE R2, [R1 + 16]	F3 (4B)	Op:31 | Rs:2 Rb:1 | Offset:0010	31 21 00 10
0x00200131	HLT	1 Byte	Op:00	00

Trazado de Verificación Técnica

- **Estado inicial en RAM (0x00202000):** A = 48 (0x30), B = 18 (0x12). R1 = 0x00202000, R2 = 48, R3 = 18.
- **Paso 1:** CMP R2, R3 (48 - 18 = 30 > 0 → Z = 0, N = 0). No salta en JZ ni en JN. Ejecuta SUB R2, R2, R3 → R2 = 30. JMP regresa a 0x00200114.
- **Paso 2:** CMP R2, R3 (30 - 18 = 12 > 0 → Z = 0, N = 0). Ejecuta SUB R2, R2, R3 → R2 = 12. JMP regresa a 0x00200114.
- **Paso 3:** CMP R2, R3 (12 - 18 = -6 < 0 → Z = 0, N = 1). Salta por JN a B_ES_MAYOR en 0x00200125. Ejecuta SUB R3, R3, R2 → R3 = 18 - 12 = 6. JMP regresa a 0x00200114.
- **Paso 4:** CMP R2, R3 (12 - 6 = 6 > 0 → Z = 0, N = 0). Ejecuta SUB R2, R2, R3 → R2 = 12 - 6 = 6. JMP regresa a 0x00200114.
- **Paso 5:** CMP R2, R3 (6 - 6 = 0 → Z = 1, N = 0). JZ detecta Z = 1 y bifurca directamente a FIN_EUCLIDES (0x0020012D).
- STORE guarda R2 = 6 (0x0000000000000006) en 0x00202010.
- Termina en HLT en 0x00200131. MCD(48, 18) = 6, verificado con exactitud.

3. Algoritmo 3: Generación de la Sucesión de Fibonacci en Memoria RAM

Descripción y Pseudocódigo

Genera de forma iterativa los primeros *N* = 7 términos de la sucesión de Fibonacci (*F*0 = 0, *F*1 = 1, *F*k+1 = *F*k + *F*k-1) y los almacena secuencialmente en palabras de 64 bits en la memoria RAM.

```
HTML
// Almacena en RAM a partir de 0x00203000
// Cantidad de terminos = 7
F0 = 0
F1 = 1
Mem[0x00203000] = F0
Mem[0x00203008] = F1

ptr = 0x00203010
contador = 5          // Restan 5 terminos por generar

mientras contador > 0 hacer:
    F_sig = F1 + F0
    Mem[ptr] = F_sig
    F0 = F1
    F1 = F_sig
    ptr = ptr + 8
    contador = contador - 1
fin mientras

// Secuencia generada en memoria: [0, 1, 1, 2, 3, 5, 8]
```

Programa en Lenguaje Ensamblador

- **Dirección base de carga del código:** 0x00200200
- **Ubicación de almacenamiento en RAM:** 0x00203000
- **Asignación de registros:**
    - **R1:** Puntero dinámico a la celda de memoria RAM (ptr)
    - **R2:** Término *F*k-1
    - **R3:** Término *F*k
    - **R4:** Contador de iteraciones restantes (5)
    - **R5:** Siguiente término generado (*F*k+1)

```
HTML
;
================================================================
========
; ALGORITMO 3: SUCESIÓN DE FIBONACCI EN RAM
```

```
;
================================================================
========
0x00200200:  ADDI  R1, R0, 0x0020     ; R1 = 0x0020
0x00200204:  SHL   R1, R1, 16         ; R1 = 0x00200000
0x00200208:  ADDI  R1, R1, 0x3000     ; R1 = 0x00203000 (Base del
arreglo)
0x0020020C:  ADDI  R2, R0, 0          ; R2 = F0 = 0
0x00200210:  ADDI  R3, R0, 1          ; R3 = F1 = 1
0x00200214:  STORE R2, [R1 + 0]       ; Mem[0x00203000] = 0 (F0)
0x00200218:  STORE R3, [R1 + 8]       ; Mem[0x00203008] = 1 (F1)
0x0020021C:  ADDI  R1, R1, 16         ; ptr = ptr + 16 (apunta a
0x00203010)
0x00200220:  ADDI  R4, R0, 5          ; R4 = 5 términos restantes
por generar

LOOP_FIBONACCI:
0x00200224:  ADD   R5, R3, R2         ; F_sig = F1 + F0
0x00200227:  STORE R5, [R1 + 0]       ; Mem[ptr] = F_sig
0x0020022B:  ADDI  R2, R3, 0          ; F0 = F1
0x0020022F:  ADDI  R3, R5, 0          ; F1 = F_sig
0x00200233:  ADDI  R1, R1, 8          ; ptr = ptr + 8 (siguiente
posición en RAM)
0x00200237:  SUBI  R4, R4, 1          ; contador = contador - 1
(actualiza Z)
0x0020023B:  JNZ   LOOP_FIBONACCI     ; Si contador != 0, repetir
(salto -26B)

0x0020023E:  HLT                      ; Fin del programa
```

Traducción Manual a Lenguaje de Máquina (Big-Endian)

Dirección	Instrucción Ensamblador	Formato	Desglose Hexadecimal (Big-Endian)	Código de Máquina (Hex)
0x00200200	ADDI R1, R0, 0x0020	F3 (4B)	Op:14 | Rd:1 Rs1:0 | Imm:0020	14 10 00 20
0x00200204	SHL R1, R1, 16	F3 (4B)	Op:24 | Rd:1 Rs1:1 | Imm:0010	24 11 00 10
0x00200208	ADDI R1, R1, 0x3000	F3 (4B)	Op:14 | Rd:1 Rs1:1 | Imm:3000	14 11 30 00
0x0020020C	ADDI R2, R0, 0	F3 (4B)	Op:14 | Rd:2 Rs1:0 | Imm:0000	14 20 00 00
0x00200210	ADDI R3, R0, 1	F3 (4B)	Op:14 | Rd:3 Rs1:0 | Imm:0001	14 30 00 01
0x00200214	STORE R2, [R1 + 0]	F3 (4B)	Op:31 | Rs:2 Rb:1 | Offset:0000	31 21 00 00
0x00200218	STORE R3, [R1 + 8]	F3 (4B)	Op:31 | Rs:3 Rb:1 | Offset:0008	31 31 00 08

Dirección	Instrucción Ensamblador	Formato	Desglose Hexadecimal (Big-Endian)	Código de Máquina (Hex)
0x0020021C	ADDI R1, R1, 16	F3 (4B)	Op:14 | Rd:1 Rs1:1 | Imm:0010	14 11 00 10
0x00200220	ADDI R4, R0, 5	F3 (4B)	Op:14 | Rd:4 Rs1:0 | Imm:0005	14 40 00 05
0x00200224	ADD R5, R3, R2	F1 (3B)	Op:10 | Rd:5 Rs1:3 | Rs2:2 0	10 53 20
0x00200227	STORE R5, [R1 + 0]	F3 (4B)	Op:31 | Rs:5 Rb:1 | Offset:0000	31 51 00 00
0x0020022B	ADDI R2, R3, 0	F3 (4B)	Op:14 | Rd:2 Rs1:3 | Imm:0000	14 23 00 00
0x0020022F	ADDI R3, R5, 0	F3 (4B)	Op:14 | Rd:3 Rs1:5 | Imm:0000	14 35 00 00
0x00200233	ADDI R1, R1, 8	F3 (4B)	Op:14 | Rd:1 Rs1:1 | Imm:0008	14 11 00 08

Dirección	Instrucción Ensamblador	Formato	Desglose Hexadecimal (Big-Endian)	Código de Máquina (Hex)
0x00200237	SUBI R4, R4, 1	F3 (4B)	Op:15 | Rd:4 Rs1:4 | Imm:0001	15 44 00 01
0x0020023B	JNZ LOOP_FIBONACCI	F4 (3B)	Op:42 | Off:0x0224 - 0x023E = -26	42 FF E6
0x0020023E	HLT	1 Byte	Op:00	00

Trazado de Verificación Técnica

- **Inicialización:**
  - Mem[0x00203000] = 0x0000000000000000 (*F*0).
  - Mem[0x00203008] = 0x0000000000000001 (*F*1).
  - Puntero R1 = 0x00203010, contador R4 = 5.
- **Iteración 1 (R4 = 5):** R5 = 1 + 0 = 1. STORE escribe 1 en 0x00203010. R2 = 1, R3 = 1, R1 = 0x00203018. SUBI deja R4 = 4 (Z = 0). JNZ salta a 0x00200224.
- **Iteración 2 (R4 = 4):** R5 = 1 + 1 = 2. STORE escribe 2 en 0x00203018. R2 = 1, R3 = 2, R1 = 0x00203020. SUBI deja R4 = 3 (Z = 0). JNZ salta a 0x00200224.
- **Iteración 3 (R4 = 3):** R5 = 2 + 1 = 3. STORE escribe 3 en 0x00203020. R2 = 2, R3 = 3, R1 = 0x00203028. SUBI deja R4 = 2 (Z = 0). JNZ salta a 0x00200224.
- **Iteración 4 (R4 = 2):** R5 = 3 + 2 = 5. STORE escribe 5 en 0x00203028. R2 = 3, R3 = 5, R1 = 0x00203030. SUBI deja R4 = 1 (Z = 0). JNZ salta a 0x00200224.
- **Iteración 5 (R4 = 1):** R5 = 5 + 3 = 8. STORE escribe 8 en 0x00203030. R2 = 5, R3 = 8, R1 = 0x00203038. SUBI deja R4 = 0 (Z = 1 en SR). JNZ no bifurca.
- Cae en 0x0020023E: HLT detiene la máquina.
- **Memoria RAM resultante verificada:**
  - 0x00203000: 0
  - 0x00203008: 1
  - 0x00203010: 1
  - 0x00203018: 2

    - 0x00203020: 3
    - 0x00203028: 5
    - 0x00203030: 8
- Generación secuencial en memoria verificada al 100%.

Verificación de la Máquina

La comprobación y el trazado de la ejecución de los tres programas propuestos demostraron que el diseño arquitectónico cuenta con las instrucciones, registros, memoria y mecanismos de control necesarios para implementar correctamente los tres algoritmos seleccionados.